

实验 7 进程通信——信号、管道和消息

一、实验目标

- (1)熟悉 openEuler 系统中进程间软中断基本原理
- (2)熟悉 openEuler 管道通信和消息传递机理
- (3)基于 openEuler 实现信号、管道和消息通信

二、背景知识

2.1 信号

2.1.1 信号的定义

每个信号都对应一个正整数常量(称为 `signal number`，即信号编号。定义在系统头文件`<signal.h>`中)，代表同一用户的诸进程之间传送信息的类型。每个进程在运行时，都要通过信号机制来检查是否有信号到达。若有，便**中断正在执行的程序**，转向与该信号相对应的处理程序，以完成对该事件的处理。处理结束后再返回到原来的断点继续执行。实质上，信号机制是对中断机制的一种模拟，故在早期 UNIX 版本中又把它称为**软中断**。

信号与中断的相似点：

- (1)采用了相同的异步通信方式；
- (2)当检测到有信号或中断请求时，都暂停正在执行的程序而转去执行相应的处理程序；
- (3)都在处理完毕后返回到原来的断点；
- (4)对信号或中断都可进行屏蔽。

信号与中断的区别：

- (1)中断有优先级，而信号没有优先级，所有的信号都是平等的；
- (2)信号处理程序是在用户态下运行的，而中断处理程序是在核心态下运行；
- (3)中断响应是及时的，而信号响应通常都有较大的时间延迟。

信号机制具有以下三方面的功能：

- (1)发送信号。发送信号的程序用系统调用 `kill()` 实现；
- (2)预置对信号的处理方式。接收信号的程序用 `signal()` 来实现对处理方式的

预置；

(3)收到信号的进程按事先的规定完成对相应事件的处理。

2.1.2 信号的发送

信号的发送是指由发送进程把信号送到指定进程的信号域的某一位上。如果目标进程正在一个可被中断的优先级上睡眠，核心便将它唤醒，发送进程就此结束。一个进程可能在其信号域中有多个位被置位，代表有多种类型的信号到达，但对于一类信号，进程却只能记住其中的某一个。

2.1.3 对信号的处理

当一个进程要进入或退出一个低优先级睡眠状态时，或一个进程即将从核心态返回用户态时，核心都要检查该进程是否已收到软中断。当进程处于核心态时，即使收到软中断也不予理睬；只有当它返回到用户态后，才处理软中断信号。对软中断信号的处理分三种情况进行：

(1)如果进程收到的软中断是一个已决定要忽略的信号（function=1），进程不做任何处理便立即返回；

(2)进程收到软中断后便退出（function=0）；

(3)执行用户设置的软中断处理程序。

2.1.4 所涉及的中断调用

kill()系统调用格式：**int kill(int pid,int sig);**

其中，pid 是一个或一组进程的标识符，参数 sig 是要发送的软中断信号。

(1)pid>0 时，内核将信号发送给进程 pid。

(2)pid=0 时，核心将信号发送给与发送进程同组的所有进程。

(3)pid=-1 时，核心将信号发送给所有用户标识符真正等于发送进程的有效用户标识号的进程。

signal()预置对信号的处理方式，允许调用进程控制软中断信号。系统调用格式：**#include int signal(int sig,void *function);**

其中，sig 用于指定信号的类型，sig 为 0 则表示没有收到任何信号，其他如下表（Linux 中使用命令“kill -l”可查看支持的信号量）：

值	符号名字	说明
1	SIGHUP	挂起（hangup）

2	SIGINT	中断，当用户从键盘按^c 键或^break 键时
3	SIGQUIT	退出，当用户从键盘按 quit 键时
4	SIGILL	非法指令
5	SIGTRAP	跟踪陷阱（trace trap），启动进程，跟踪代码的执行
6	SIGIOT	IOT 指令
7	SIGEMT	EMT 指令
8	SIGFPE	浮点运算溢出
9	SIGKILL	杀死、终止进程
10	SIGBUS	总线错误
11	SIGSEGV	段违例（segmentation violation），进程试图去访问其虚地址空间以外的位置
12	SIGSYS	系统调用中参数错，如系统调用号非法
13	SIGPIPE	向某个非读管道中写入数据
14	SIGALRM	闹钟。当某进程希望在某时间后接收信号时发此信号
15	SIGTERM	软件终止（software termination）
16	SIGUSR1	用户自定义信号 1
17	SIGUSR2	用户自定义信号 2
18	SIGCLD	某个子进程死
19	SIGPWR	电源故障

function 指的是在该进程中的一个函数入口地址。在核心返回用户态时，它以软中断信号的序号作为参数调用该函数，对除了信号 SIGKILL，SIGTRAP 和 SIGPWR 以外的信号，核心自动地重新设置软中断信号处理程序的值为 SIG_DFL，一个进程不能捕获 SIGKILL 信号。function 的解释如下：

- (1)function=1 时，进程对 sig 类信号不予理睬，亦即屏蔽了该类信号；
- (2)function=0 时，缺省值，进程在收到 sig 信号后应终止自己；
- (3)function 为非 0, 非 1 类整数时，function 的值即作为信号处理程序的指针。

2.2 管道

2.2.1 管道的定义

所谓管道，是指能够连接一个写进程和一个读进程的、并允许它们以生产者

—消费者方式进行通信的一个共享文件，又称为 **pipe** 文件。由写进程从管道的写入端（句柄 1）将数据写入管道，而读进程则从管道的读出端（句柄 0）读出数据。

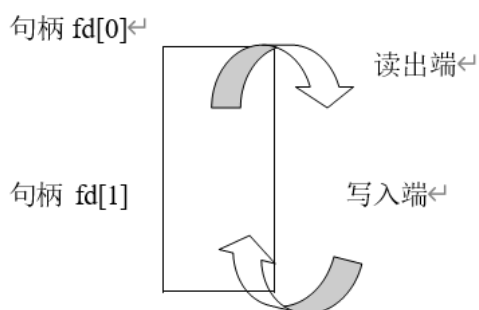


图 7.1 管道

2.2.2 管道的类型

(1)**有名管道**：一个可以在文件系统中长期存在的、具有路径名的文件。用系统调用 `mknod()` 建立。它克服无名管道使用上的局限性，可让更多的进程也能利用管道进行通信。因而其它进程可以知道它的存在，并能利用路径名来访问该文件。对有名管道的访问方式与访问其他文件一样，需先用 `open()` 打开。

(2)**无名管道**：一个临时文件。利用 `pipe()` 建立起来的无名文件（无路径名）。只用该系统调用所返回的文件描述符来标识该文件，故只有调用 `pipe()` 的进程及其子孙进程才能识别此文件描述符，才能利用该文件（管道）进行通信。当这些进程不再使用此管道时，核心收回其索引结点。

2.2.3 所涉及的系统调用

`pipe()` 建立无名管道，系统调用格式：`#include int pipe(int filedes[2]);` 其中，`filedes[1]` 是写入端文件描述符，`filedes[0]` 是读出端文件描述符。

`read()` 系统调用格式：`#include int read(int fd, char *buf, unsigned nbyte);` 功能为从 `fd` 所指示的文件中读出 `nbyte` 个字节的数据，并将它们送至由指针 `buf` 所指示的缓冲区中。如该文件被加锁，等待，直到锁打开为止。

`write()` 系统调用格式：`#include int write(int fd, char *buf, unsigned nbyte);` 功能为把 `nbyte` 个字节的数据，从 `buf` 所指向的缓冲区写到由 `fd` 所指向的文件中。如文件加锁，暂停写入，直至开锁。

2.3 消息

2.3.1 消息的定义

消息（message）是一个格式化的可变长的信息单元。消息机制允许由一个进程给其它任意的进程发送一个消息。当一个进程收到多个消息时，可将它们排成一个消息队列。消息使用二种重要的数据结构：一是消息首部，其中记录了一些与消息有关的信息，如消息数据的字节数；二是消息队列头表，其每一表项是作为一个消息队列的消息头，记录了消息队列的有关信息。

2.3.2 消息机制的数据结构：

(1)消息首部：记录一些与消息有关的信息，如消息的类型、大小、指向消息数据区的指针、消息队列的链接指针等。

(2)消息队列头表：其每一项作为一个消息队列的消息头，记录了消息队列的有关信息如指向消息队列中第一个消息和指向最后一个消息的指针、队列中消息的数目、队列中消息数据的总字节数、队列所允许消息数据的最大字节总数，还有最近一次执行发送操作的进程标识符和时间、最近一次执行接收操作的进程标识符和时间等。

(3)消息队列描述符：UNIX 中，每一个消息队列都有一个称为关键字（key）的名字，是由用户指定的；消息队列有一消息队列描述符，其作用与用户文件描述符一样，也是为了方便用户和系统对消息队列的访问。

2.3.3 所涉及的系统调用

`msgget()`：创建一个消息队列，获得一个消息队列描述符。核心将搜索消息队列头表，确定是否有指定名字的消息队列。若无，核心将分配一新的消息队列头，并对它进行初始化，然后给用户返回一个消息队列描述符，否则它只是检查消息队列的许可权便返回。

系统调用格式：`msgqid=msgget(key,flag);`

参数定义：`int msgget(key,flag)`

`key_t key;`

`int flag;`

其中：`key` 是用户指定的消息队列的名字；

`flag` 是用户设置的标志（控制命令）和访问方式，如：

`IPC_CREAT | 0400` 该队列是否已被创建，无则创建，是则打开；

IPC_EXCL | 0400 该队列的创建是互斥的，若已创建则失败；

msgqid 是该系统调用返回的描述符，失败则返回-1。

msgsnd(): 发送消息。向指定的消息队列发送一个消息，并将该消息链接到该消息队列的尾部。

系统调用格式: msgsnd(msgqid,msgp,size,flag)

参数定义: int msgsnd(msgqid,msgp,size,flag)

int msgqid,size,flag;

struct msgbuf * msgp;

其中 msgqid 是 msgget 返回的消息队列描述符；msgp 是指向用户消息缓冲区的一个结构体指针。

size 指示由 msgp 指向的数据结构中字符数组的长度；即消息的长度。这个数组的最大值由 MSG_MAX()系统可调用参数来确定。

flag 规定当核心用尽内部缓冲空间时应执行的动作。进程是等待，还是立即返回。若在标志 flag 中未设置 IPC_NOWAIT 位，则当该消息队列中的字节数超过最大值时，或系统范围的消息数超过某一最大值时，调用 msgsnd 进程睡眠。若是设置 IPC_NOWAIT，则在此情况下，msgsnd 立即返回。

msgrcv(): 接受消息。从指定的消息队列中接收指定类型的消息。

系统调用格式: msgrcv(msgqid,msgp,size,type,flag)

参数定义: int msgrcv(msgqid,msgp,size,type,flag)

int msgqid,size,flag;

struct msgbuf *msgp;

long type;

其中，msgqid,msgp,size,flag 与 msgsnd 中的对应参数相似；type 是规定要读的消息类型。flag 规定倘若该队列无消息，核心应做的操作。如此时设置了 IPC_NOWAIT 标志，则立即返回，若在 flag 中设置了 MS_NOERROR 且所接收的消息大于 size，则核心截断所接收的消息。

msgctl(): 消息队列的操纵。读取消息队列的状态信息并进行修改，如查询消息队列描述符、修改它的许可权及删除该队列等。

系统调用格式: msgctl(msgqid,cmd,buf);

```
参数定义:  int msgctl(msgqid,cmd,buf);  
            int msgqid,cmd;  
            struct msgqid_ds *buf;
```

其中，函数调用成功时返回 0，不成功则返回-1。buf 是用户缓冲区地址，供用户存放控制参数和查询结果；cmd 是规定的命令。命令可分三类：

(1)IPC_STAT 查询有关消息队列情况的命令。如查询队列中的消息数目、队列中的最大字节数、最后一个发送消息的进程标识符、发送时间等，结果在 buf 中；

(2)IPC_SET 按 buf 指向的结构中的值，设置和改变有关消息队列属性的命令。如 改变消息队列的用户标识符、消息队列的许可权等；

(3)IPC_RMID 从消息队列头表中删除该消息队列。

三、实验步骤

3.1 实验任务 1

编写程序：用 fork() 创建两个子进程，再用系统调用 signal() 让父进程捕捉键盘上来的中断信号（即按 Ctrl+c 键）；捕捉到中断信号后，父进程用系统调用 kill() 向两个子进程发出信号，子进程捕捉到信号后分别输出下列信息后终止：

Child process1 is killed by parent!

Child process2 is killed by parent!

父进程等待两个子进程终止后，输出如下的信息后并写入本地文件后终止

Parent process is killed!

(1)在命令行中执行命令 touch signal.c 创建.c 文件，输入命令 vi signal.c 进入编辑模式：

```
[root@localhost ~]# touch signal.c  
[root@localhost ~]# vi signal.c
```

(2)点击键盘上“i”按键开始写入，写入代码如下：

```
#include <signal.h>  
#include <unistd.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/wait.h>  
#include <string.h>  
void waiting();
```

```

void stop();
int wait_mark;
int *fp;
void waiting() {
    while(wait_mark!=0);
}
void stop() {
    wait_mark=0;
}
void main() {
    char file[20];
    int p1,p2;
    printf("please input the file name:\n");
    scanf("%s",file);
    fp = fopen(file,"w+");
    signal(SIGINT,SIG_IGN);
    while((p1=fork())!=-1);
    if(p1>0) {
        while((p2=fork())!=-1);
        if(p2>0) {
            printf("请按 CTRL+C 终止程序\n");
            wait_mark=1;
            signal(SIGINT,stop);
            waiting();
            kill(p1,10);
            kill(p2,12);
            wait(0);
            wait(0);
            fprintf(fp,"parent process is killed\n");
            exit(0);
        } else {
            wait_mark=1;
            signal(12,stop);
            waiting();
            lockf(*fp,1,0);//lock
            printf("child 2 prcess is killed by parent\n");
            fprintf(fp,"child 2 prcess is killed by parent\n");
            lockf(*fp,0,0);//unlock
            exit(0);
        }
    } else {
        wait_mark=1;
        signal(10,stop);
        waiting();
    }
}

```



```

        lockf(*fp,1,0);//lock
        printf("child 1 process is killed by parent\n");
        fprintf(fp,"child 1 process is killed by parent\n");
        lockf(*fp,0,0);//unlock
        exit(0);
    }
}

```

(3)点击键盘上“Esc”键，输入“:wq”并回车，退出 vi。

(4)输入命令 `gcc signal.c -o signal` 编译.c 文件，输入命令 `./signal` 执行：

```

[root@localhost ~]# gcc signal.c -o signal
[root@localhost ~]# ./signal

```

(5)实验结果应如下所示：

```

[root@localhost ~]# ./signal
please input the file name:
test.txt
请按CTRL+C终止程序
^Cchild 2 prcess is killed by parent
child 1 process is killed by parent
[root@localhost ~]# cat test.txt
child 2 prcess is killed by parent
child 1 process is killed by parent
parent process is killed

```

图 7.2 任务 1 实验结果

3.2 实验任务 2

编写一段程序，实现进程的管道通信。使用 `pipe()` 建立一条管道线。两个子进程 p1 和 p2 分别向管道各写一句话：

Child 1 is sending message!

Child 2 is sending message!

而父进程则从管道中分别读出来自于两个子进程的信息，显示在屏幕上。

(1)在命令行中执行命令 `touch pipe.c` 创建.c 文件，输入命令 `vi pipe.c` 进入编辑模式：

```

[root@localhost ~]# touch pipe.c
[root@localhost ~]# vi pipe.c

```

②点击键盘上“i”按键开始写入，写入代码如下：

```

#include<unistd.h>

```

```

#include<signal.h>
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h>
int pid1,pid2; //存储两个子进程的 PID 号;
int main() {
    int fd[2]; //打开文件的文件描述符
    char OutPipe[100],InPipe[100]; //存储要写入管道的字符串
    pipe(fd); //创建管道---建立一无名管道
    while((pid1 = fork()) == -1); //创建子进程
    if(pid1 == 0) { //子进程
        lockf(fd[1],1,0); //lockf(fd,1,0)给文件上锁，使实现互斥访问
        //将字符串输出到 OutPipe(目的字符串)中
        sprintf(OutPipe,"Child 1 is sending message!");
        //从管道的写入端（句柄 1）将 50 个字节数据写入管道
        write(fd[1],OutPipe,50);
        lockf(fd[1],0,0); //lockf(fd,0,0)是文件解锁
        sleep(1); //延时 1 秒
        exit(0); //子进程结束
    } else {
        while((pid2 = fork()) == -1); //创建子进程
        if(pid2 == 0) {
            sleep(0);
            lockf(fd[1],1,0); //lockf(fd,1,0)给文件上锁，使实现互斥访问
            //将字符串输出到 OutPipe(目的字符串)中
            sprintf(OutPipe,"Child 2 is sending message!");
            //从管道的写入端（句柄 1）将 50 个字节数据写入管道
            write(fd[1],OutPipe,50);
            lockf(fd[1],0,0); //lockf(fd,0,0)是文件解锁
            sleep(1);
            exit(0); //子进程结束
        } else { //父进程
            wait(0); //等待子进程执行完毕
            wait(0); //等待子进程执行完毕
            read(fd[0],InPipe,50); //从管道的读出端（句柄 0）读出 50 个字节数据
            printf("%s\n",InPipe); //父进程将字符串显示在屏幕上
            read(fd[0],InPipe,50);
            printf("%s\n",InPipe);
            exit(0);
        }
    }
    return 0;
}

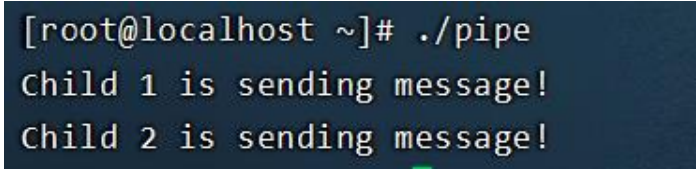
```

(3)点击键盘上“Esc”键，输入“:wq”并回车，退出 vi。

(4)输入命令 `gcc pipe.c -o pipe` 编译.c 文件, 输入命令 `./pipe` 执行,查看输出结果:

```
[root@localhost ~]# gcc pipe.c -o pipe
[root@localhost ~]# ./pipe
```

(5)实验结果应如下所示:



```
[root@localhost ~]# ./pipe
Child 1 is sending message!
Child 2 is sending message!
```

图 7.3 任务 2 实验结果

3.3 实验任务 3

编写一个程序,使用系统调用 `msgget()`, `msgsnd()`, `msgrcv()`, `msgctl()`实现消息的发送和接收,一个子进程发送 10 条消息,一个子进程接收 10 条消息。(MSGKEY 取值 75, 消息的长度取 1024)

(1)在命令行中执行命令 `touch msgexch.c` 创建.c 文件,输入命令 `vi msgexch.c` 进入编辑模式:

```
[root@localhost ~]# touch msgexch.c
[root@localhost ~]# vi msgexch.c
```

(2)点击键盘上“i”按键开始写入, 写入代码如下:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/msg.h>
#include <sys/ipc.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#define MSGKEY 75
struct msgform {
    long mtype;
    char mtrex[1030];
}msg;
int msgqid,i;

int CLIENT() {
    int i;
    msgqid=msgget(MSGKEY,0777);
    for(i=10;i>=1;i--) {
```

```

        msg.mtype=i;
        printf("(client)sent\n");
        msgsnd(msgqid,&msg,1024,0);
    }
    exit(0);
}

int SERVER() {
    msgqid=msgget(MSGKEY,0777|IPC_CREAT);
    do {
        msgrcv(msgqid,&msg,1030,0,0);
        printf("(server)received\n");
    }while(msg.mtype!=1);
    msgctl(msgqid,IPC_RMID,0);
    exit(0);
}

int main() {
    while((i=fork())!=-1);
    if(!i)
        SERVER();
    while((i=fork())!=-1);
    if(!i)
        CLIENT();
    wait(0);
    wait(0);
    return 0;
}

```

(4)输入命令 gcc msgexch.c -o msgexch 编译.c 文件, 输入命令 ./msgexch 执行, 查看输出结果:

```

[root@localhost ~]#gcc msgexch.c -o msgexch
[root@localhost ~]# ./msgexch

```

(5)实验结果应如下所示:

```
[root@localhost ~]# ./process
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(client)sent
(server)received
(server)received
(server)received
(server)received
(server)received
(server)received
(server)received
(server)received
(server)received
(server)received
```

图 7.4 任务 3 实验结果

四、实验讨论

(1) 在第 1 个任务中，为什么要使用信号和 kill 系统调用来实现父进程与子进程之间的通信？有哪些其他的进程间通信方式可以替代信号和 kill 系统调用？

(2) 在第 2 个任务中，管道是如何实现进程间通信的？管道有什么限制和局限性？

(3) 在第 3 个任务中，消息队列是如何实现进程间通信的？相比于其他进程间通信方式，消息队列有什么优势和劣势？